

Architecture Specification: Direct Snowpark Integration for Snowflake Cortex

Document Type: Technical Implementation & Justification

Target Environment: Ciena Corporate Network / Digital Experience Group

Target Technology: Snowflake Snowpark Python API, Snowflake Cortex, Okta SSO, Zscaler Proxy

1. Executive Summary

This document outlines the technical specifications and architectural justification for transitioning Snowflake Cortex access from a Managed Model Context Protocol (MCP) server middleware to a direct, native integration using the `snowflake-snowpark-python` library. This new paradigm leverages Okta Single Sign-On (SSO) for identity management and custom certificate injection to securely traverse corporate SSL inspection proxies, establishing a low-latency, resilient pipeline for AI and data automation workloads.

2. Technical Justification

2.1 Elimination of Middleware Abstraction

The previous architecture relied on an MCP server to translate conversational JSON payloads into Snowflake-compatible REST API calls. While effective for simple chat interfaces, this middleware introduces latency, restricts execution flexibility, and complicates debugging. Connecting directly via Snowpark eliminates this intermediary, reducing point-of-failure risks and allowing Python applications to execute native DataFrames and SQL commands instantaneously.

2.2 Native Cortex Execution via Snowpark

Snowpark enables direct execution of machine learning workloads on Snowflake's compute nodes. By utilizing the `Session.sql()` method, automation scripts can directly invoke functions such as `SNOWFLAKE.CORTX.COMPLETE()` alongside standard database queries. This permits seamless integration of generative AI summarization directly into data processing pipelines (e.g., batch-updating Excel case logs) without routing data through external API gateways.

2.3 Identity & Access Management Compliance

The direct Snowpark integration does not require hardcoded passwords, service account API keys, or long-lived static tokens. By utilizing the `externalbrowser` authenticator, the

application hands the authentication handshake off to the organization's Okta SAML flow. This ensures that all corporate Conditional Access Policies, Multi-Factor Authentication (MFA) requirements, and active session verifications are strictly enforced. The script operates securely within the exact permission boundaries of the authenticated user's profile.

2.4 Resilient Proxy Traversal (Zscaler SSL Inspection)

Corporate networks utilizing deep packet inspection (e.g., Zscaler) inherently intercept external SSL connections. Standard Python libraries reject these intercepted connections as potential Man-in-the-Middle (MITM) attacks, causing scripts to hang or fail with

`certificate verify failed` errors. This architecture solves this natively via two mechanisms:

1. **Certificate Bundle Injection:** Overriding the default Python certificate authority (CA) store with a unified `.pem` bundle that includes both public web authorities and the internal Zscaler root certificate.
2. **OCSP Disablement:** Bypassing background Online Certificate Status Protocol (OCSP) checks which are frequently blocked or silently dropped by corporate proxies, preventing infinite hang states during the initial handshake.

3. Implementation Specifications

3.1 Environment Prerequisites

The environment must be configured with the Snowpark framework rather than the standard Snowflake SQL connector.

Bash



```
pip install snowflake-snowpark-python
```

3.2 Certificate Management Configuration

To ensure the underlying `requests` library trusts the corporate proxy, the operating system environment must be pointed to the combined certificate bundle *before* the Snowpark session is initialized. This bundle must contain standard CA roots appended with the Zscaler root certificate.

```
import os
os.environ["REQUESTS_CA_BUNDLE"] = r"C:\path\to\combined_ca_bundle.pem"
```

3.3 Connection Parameter Definitions

The following parameters dictate the routing, authentication, and security overrides required to establish the Snowpark session within the Ciena environment.

Parameter	Value	Functional Purpose
account	ciena-ciena	Directs the connection to the specific corporate Snowflake tenant.
user	<user>@CIENA.COM	The explicit identity claim passed to Okta for SAML routing.
authenticator	externalbrowser	Triggers the local temporary web server to catch the Okta SAML assertion.
role	SSO_SNOWFLAKE_GAI_SVC_RO	Enforces the read-only AI services role for executing Cortex functions.
warehouse	GAI_POC	Specifies the compute cluster dedicated to processing the AI workloads.
database	FLYGAIP	The target database containing the AI service models.
schema	GAI_SERVICES	The specific schema housing the deployed Cortex

Parameter	Value	Functional Purpose
		integration.
<code>insecure_mode</code>	<code>True</code>	Instructs the Snowflake driver to accept the local proxy certificates.
<code>disable_ocsp_checks</code>	<code>True</code>	Prevents the driver from silently hanging while attempting unreachable OCSP validations.
...		

4. Reference Implementation

The following boilerplate script demonstrates the complete, secure initialization of the Snowpark session and the subsequent execution of a Snowflake Cortex text-completion workload.

Python



```
import os
import getpass
from snowflake.snowpark import Session

# 1. Inject Unified Certificate Bundle for Proxy Traversal
os.environ["REQUESTS_CA_BUNDLE"] = r"C:\path\to\combined_ca_bundle.pem"

def execute_cortex_workload():
    # 2. Dynamic User Resolution
    current_user = getpass.getuser().upper() + "@CIENA.COM"

    # 3. Session Configuration
    connection_parameters = {
        "account": "ciena-ciena",
        "user": current_user,
        "authenticator": "externalbrowser",
        "role": "SSO_SNOWFLAKE_GAI_SVC_RO",
        "warehouse": "GAI_POC",
        "database": "FLYGAIP",
        "schema": "GAI_SERVICES",
```

```

        "insecure_mode": True,
        "disable_ocsp_checks": True
    }

    try:
        # 4. Initialize Authenticated Session
        session = Session.builder.configs(connection_parameters).create()

        # 5. Define Workload Payload
        log_data = "14:02 EST - ALARM: Loss of Signal (LOS) on Slot 3, Port 1. Tech d
        prompt = f"Summarize this network outage event in one short sentence: {log_da

        # 6. Execute Native Cortex Function
        query = f"SELECT SNOWFLAKE.CORTEX.COMPLETE('llama3.1-8b', '{prompt}') AS AI_R
        results = session.sql(query).collect()

        # 7. Process Output
        print(results[0]["AI_RESPONSE"])

    except Exception as e:
        print(f"Snowflake integration failure: {e}")
    finally:
        # 8. Resource Management
        if 'session' in locals():
            session.close()

if __name__ == "__main__":
    execute_cortex_workload()

```